

skbuild: Extracting Kbuild Build Conditions via Eager-Merge Symbolic Execution

ANONYMOUS AUTHOR(S), Anonymous Institution, USA

Large configurable systems such as the Linux kernel, BusyBox, and coreboot describe what gets compiled with Kbuild-style Makefiles: a per-directory `obj-$(CONFIG_X)`-shaped vocabulary layered on GNU Make, driven by a Kconfig-defined space of Boolean and tristate options. A single build under one `.config` answers only which files were selected for *that* configuration; it does not say how the build varies as options change, which is what a developer needs to explain a crash under an unfamiliar configuration, to bound the effect of flipping one option, or to identify files a Kbuild refactor silently stopped building. We present skbuild, a symbolic-execution engine for the Kbuild-relevant subset of GNU Make that computes, for every object file, a Z3-checkable Boolean condition over Kconfig symbols describing exactly when that file participates in the build. Following a fork-per-branch design that forks a new execution path at every `ifeq/ifdef` makes real Kbuild trees intractable — a 30-line synthetic fixture already produces dozens of forked paths, and a first attempt at the checked-in BusyBox snapshot exhausts the pattern entirely before finishing — skbuild instead keeps a single symbolic state per Kbuild file, representing each Make variable as a set of (word, condition) pairs, and merges a conditional's two branches back into that one state immediately, gating each branch's contribution by its own guard. This design is directly adapted from Cybolic's eager-merge representation for CMake [1], applied here to a build-description language with a different, and in some ways simpler, variability model: Kbuild's symbolic inputs are Kconfig's small, explicitly finite Boolean and tristate domains rather than CMake's unbounded-string `option()` values. We evaluate skbuild on [TODO: BusyBox / coreboot / Linux, N pinned releases, M reconciled object sets, K categorized mismatches].

CCS Concepts: • **Software and its engineering** → **Software verification**; *Software testing and debugging*.

Additional Key Words and Phrases: build systems, Kbuild, Linux kernel, symbolic execution, build conditions, SMT solving, variability

ACM Reference Format:

Anonymous Author(s). 2027. skbuild: Extracting Kbuild Build Conditions via Eager-Merge Symbolic Execution. *Proc. ACM Softw. Eng.* 1, FSE, Article 1 (July 2027), 6 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Large, configurable C codebases — the Linux kernel, BusyBox, coreboot, and other projects that adopted the Kbuild convention — do not compile a fixed set of source files. A per-directory Makefile or Kbuild file assigns object names to variables such as `obj-y`, `obj-m`, and `lib-y` conditionally on Kconfig options (`obj-$(CONFIG_USB) += usb.o`), nests those assignments inside `ifeq/ifdef` blocks, and recurses into subdirectories whose membership is itself conditional. Kconfig, in turn, defines each `CONFIG_*` symbol's domain: most commonly Boolean (y/not-set) or tristate (y/m/not-set), sometimes a small enumerated or numeric range. A single invocation of `make` with one `.config` answers a narrow question — which files were selected for *that* configuration — but does not directly answer how the build varies across the configuration space the project actually ships.

Author's Contact Information: Anonymous Author(s), Anonymous Institution, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2027/7-ART1

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

That broader question matters in practice. A crash report from a shipped kernel build names a source file, but that file may or may not have been compiled in depending on options the reporter never mentioned; knowing *which* configurations could have produced the crashing binary narrows root-cause search before any other analysis starts. A patch that touches a file guarded behind `CONFIG_DEBUG_INFO` needs a build configuration under which the file is even reachable before static analysis of the patch can begin. A maintainer refactoring a Kbuild tree wants to know whether the refactor changed which files are selected under any configuration, not just under whichever one happens to be ‘make defconfig’. All three reduce to the same question this paper answers: *given an object file, under what Boolean condition over Kconfig symbols is it selected by the build?*

Nguyen and Nguyen [3] showed that this question is answerable by symbolic execution of Kbuild Makefiles specifically, treating `CONFIG_*` variables as finite-domain symbolic inputs and Z3 as the underlying solver. This paper’s skbuild is a from-scratch, from-first-principles reimplement of that direction: it keeps the CONFIG-symbols-as-symbolic- inputs framing and the Z3 backend, but replaces a fork-per-branch execution strategy — which we found, empirically, explodes even on small synthetic fixtures and cannot complete on a real multi-directory tree at all — with an eager branch-and-merge representation adapted from Cybolic, a recent system that solved the structurally identical problem (extract a per-file build condition via symbolic execution of a configuration-driven build description) for CMake [1]. Kbuild’s variability model is, in one concrete way, simpler than CMake’s: Kconfig options are explicitly finite-domain by construction (Boolean or tristate, occasionally a small enumerated set), where CMake’s `option()/cache` variables are unbounded strings that Cybolic must itself constrain to a finite domain before Z3 can reason about them. Kbuild’s variability model is, in another way, its own challenge: recursive directory traversal is itself conditional, so the same eager-merge idea has to apply not just to variable values within one file but to *which files get analyzed at all*.

Contributions. In summary, this paper makes the following contributions:

- We identify, with a concrete measurement on a small synthetic fixture and a real-world corpus, that fork-per-branch symbolic execution of Kbuild Makefiles is not viable at real-project scale (Section 2).
- We adapt Cybolic’s eager branch-and-merge symbolic-state representation [1] to Kbuild/GNU Make, including two Kbuild-specific correctness subtleties — conditionally-named `:=` overwrites, and re-gating carried-forward (untouched) values by branch guard at merge time — that do not arise in CMake’s simpler assignment model (Section 3).
- We implement this design in skbuild, a pure-Python, Z3-backed analyzer with no other runtime dependency (Section 4).
- We evaluate skbuild on BusyBox, coreboot, and the Linux kernel, reconciling predicted object sets against real, independently obtained build output and categorizing every mismatch (Section 4).

Tool and evaluation artifacts [\[TODO: artifact URL\]](#).

2 Overview

2.1 A Worked Example

Figure 1 shows a small Makefile exercising the constructs this paper is about: Boolean-guarded object selection (`obj-$(CONFIG_1)`), an unconditional overwrite whose *target variable name* is itself conditional (`obj-$(CONFIG_A) := 1.o`), and a variable (BITS) reassigned unconditionally by both arms of an `ifeq/else`.

A fork-per-branch symbolic executor treats every `ifeq` as a point where execution splits into two independent copies of the current state, one per branch, carried forward until (if ever) they are

```

99  obj-$(CONFIG_1) += 1.o
100 obj-$(CONFIG_A) := 1.o          # overwrites obj-y's value *when CONFIG_A=y*
101 obj-$(CONFIG_A) += 3.o
102 obj-$(CONFIG_B) += 3.o
103
104 ifeq ($(CONFIG_A),y)
105     BITS := 32
106 else
107     BITS := 64
108 endif
109 obj-y += probe$(BITS).o
110

```

Fig. 1. A worked example. `obj-y`'s final value for `1.o` is `CONFIG_A=y OR (CONFIG_1=y AND CONFIG_A!=y)`: the first `+=` contributes `1.o` whenever `CONFIG_1=y`, and the later `:=` only erases that contribution in the universe where it actually fires, i.e. `CONFIG_A=y`. `BITS` ends with exactly two values, 32 under `CONFIG_A=y` and 64 otherwise — `0` does not survive as a spurious third case, even though it would if the merge at the `ifeq` naively unioned each branch's raw carried-forward state.

recombined. Under this design, the seven statements in Figure 1 already fork 32 distinct execution paths in our reimplement of that strategy (Section 4.1), and the checked-in BusyBox snapshot's top-level tree does not finish analysis at all under it.

skbuild instead keeps exactly one symbolic state per Kbuild file throughout. Each Make variable's value is a map from word to the Z3 condition under which that word is currently a member (Section 3.1). A conditional clones this one state per branch, runs each branch's statements against its own clone, and immediately folds both clones back into the original state before the next statement (Section 3.2) — so state size stays proportional to program size, not to the number of branches taken so far.

2.2 Design Summary

skbuild has four pieces: a GNU Make parser (a Python 3 port of Mozilla's `pymake`), a symbolic evaluator over the guarded-value state described above, a Z3-backed condition solver, and a directory-traversal layer that recurses into subdirectories under their own guard, using the same eager-merge state to decide which subdirectories to visit at all. Recipes and rules are parsed but never executed; `shell` is disabled by default (Section 4.6 discusses this limitation). The tool reports one record per selected object: a path, a target kind (built-in / module / library), and a Boolean condition over Kconfig symbols.

3 Design

3.1 Representing Branch-Dependent Values

A Make variable in skbuild (`ds.VarG`) is a name, a flavor (`RECURSE` for `=/define`, `SIMPLY` for `:=/: :=/+`), and a map from word to Z3 condition. A symbolic state (`ds.BaseState`) is a map from variable name to `VarG`. This is the direct Kbuild analogue of Cybolic's `Switch/Const/Concat` value representation for CMake strings and lists [1], specialized to the fact that a Kbuild variable's value is, in every construct this paper's supported subset covers, a whitespace-separated set of independently-conditioned words rather than an order-sensitive string.

An assignment updates this state under an ambient guard — the conjunction of every enclosing conditional's branch condition — threaded as an explicit parameter through statement execution

rather than stored per-path. A += accumulates: each newly assigned word's condition is OR'd with the guard into its existing entry, or created fresh. A := or = overwrites, but — and this is the first of two correctness subtleties that do not arise in CMake's simpler set() — only *within* the guard under which this particular assignment's name and value actually resolved. Kbuild's obj-\$(CONFIG_A) := 1.o example in Figure 1 resolves to the variable literally named obj-y only when CONFIG_A=y; outside that condition, this statement assigns to obj-undef (a name nothing reports) and must leave obj-y's prior value untouched. skbuild's overwrite therefore computes, per pre-existing word w with condition c : $c \wedge \neg \text{cond}$ carried forward (the assignment did not fire), unioned with cond for each newly assigned word, where cond is this specific assignment's fully-resolved condition (guard $\wedge$ name match $\wedge$ value match).

3.2 Branch Execution and Eager State Merging

A ConditionBlock clones the current state twice, executes the then branch's statements against one clone under guard $\wedge$ if_cond, the else branch's statements against the other under guard $\wedge \neg$ if_cond, and merges. The second correctness subtlety — again, one with no CMake analogue, because CMake's if()/else() does not have Kbuild's pattern of an unconditional-within-the-branch overwrite whose *reachability* is what's conditional — is that merging cannot simply union each branch's raw per-word conditions computed by Section 3.1's overwrite rule. Consider Figure 1's BITS: entering the ifeq, BITS has no prior value; each branch's := computes a fresh, branch-guard-scoped condition for its own word (32 under CONFIG_A=y, 64 under its negation) — unremarkable. But if BITS had already been set to 0 before this ifeq (as it is, from an earlier ifeq \$(CONFIG_B),y) BITS := 0 endif, in the fuller version of this fixture we use for testing), each branch's own overwrite rule computes a *carry-forward* term for 0 of “CONFIG_B=y AND this branch's assignment did not fire” — and since the two branches' “did not fire” conditions are exact complements of each other, naively OR-ing the two branches' raw 0 terms together reconstructs CONFIG_B=y unconditionally, resurrecting a value that both branches, between them, always overwrite. The fix is to re-gate each branch's *entire* per-word contribution by that branch's own guard at merge time, not just rely on the per-statement conditions computed inside the branch:

$$\text{merged}(w) = (\text{guard}_{\text{then}} \wedge \text{then_state}(w)) \vee (\text{guard}_{\text{else}} \wedge \text{else_state}(w))$$

which is exactly what Cybolic's own merge_branches does for CMake values [1]; we did not initially replicate this and had to find the bug by hand-deriving Figure 1's expected conditions and comparing against skbuild's output (Section 4.1 discusses this as a methodology point, not just a bug-fix anecdote). Applying this re-gating to *every* variable at every conditional, however, grows condition expressions unboundedly across a file regardless of relevance: an untouched variable's condition would be re-wrapped in $(\text{guard}_{\text{then}} \wedge \cdot) \vee (\text{guard}_{\text{else}} \wedge \cdot)$ at every subsequent conditional even though that expression is always tautologically equal to the untouched value. skbuild tracks, per state, the set of variable names actually reassigned since the state was cloned (State.touched); merge only rebuilds names in the union of both branches' touched sets, and leaves everything else in the destination state exactly as it was — mirroring Cybolic's own changed_vars restriction to the same end [1].

3.3 Recursive Traversal

Kbuild's obj-\$(CONFIG) pattern applies to subdirectory names too (obj-\$(CONFIG_USB) += usb/); skbuild's BaseState aggregates, across every non-target, non-ignored variable in the final state, a map from subdirectory to the Z3 condition under which it is reached, OR-ing conditions where

more than one variable names the same subdirectory. Today this traversal is a straightforward per-directory recursion; it does not yet model ARCH/CROSS_COMPILE or derive root-directory reachability from the real root Makefile rather than a curated top-directory list (Section 4.6).

4 Evaluation

4.1 RQ0 (Methodology): Fork-Per-Branch Does Not Scale

[TODO: This subsection reports the actual measurement instead of the anecdotal numbers used in Section 2: re-run the retired fork-per-branch implementation (available in git history before this paper's rewrite) against Figure 1 and the BusyBox/Linux snapshots, and report path counts / wall time / where it fails outright, as a controlled before/after rather than a one-off observation.]

4.2 RQ1: Applicability

[TODO: Does skbuild run to completion on a pinned BusyBox release, a pinned coreboot release, and a pinned Linux kernel release, per PLAN.md's M1–M3 acceptance criteria? Report per-project: source LOC, number of Makefile/Kbuild files, wall time, and completion status.]

4.3 RQ2: Coverage

[TODO: Per-construct usage census across BusyBox/coreboot/Linux (PLAN.md's M4), classified the way Cybolic's RQ2 classifies CMake commands: (i) *modeled*, (ii) *correctly out of scope* (affects recipes, linking, or installation, not which files are selected — a no-op is provably harmless), or (iii) *deliberately unsupported*, reported with real usage-frequency counts rather than a bare "N warnings" number, so a reader can see whether an unmodeled construct is a corner case or, as with Cybolic's `get_filename_component` finding, the single most frequently invoked construct across every project evaluated. shell probes of the host/toolchain are the leading candidate for that role in Kbuild; state plainly whether that turned out true.]

4.4 RQ3: Performance

[TODO: Wall time, peak RSS, and Z3 call counts across the three projects, per PLAN.md's M4 instrumentation. State explicitly whether skbuild's dominant cost is Z3 solver calls under deep conditional nesting or something else — do not assume it mirrors Cybolic's string/list concatenation bottleneck without measuring.]

4.5 RQ4: Agreement with Real Builds

This is the paper's central empirical result. For each project, we use Z3 to extract, from skbuild's own reported conditions, a small set of concrete witness configurations that between them are predicted to cover every reported object (a greedy set cover over conditions, the same technique underlying Cybolic's sufficient-CI-matrix result [1]), plus at least one negative witness per file where feasible. We materialize each witness as a real `.config`, run the project's actual build against it in a clean checkout, and compare the real build's object/module output against skbuild's prediction for that same witness. [TODO: Report, per project: number of witnesses, and true/false positive/negative object counts, with every non-match causally categorized (missing generated file, unsupported construct, genuine skbuild bug, or a witness Kconfig itself would reject) per PLAN.md's M1.0 tool — not a bare diff count.]

4.6 Discussion and Threats to Validity

skbuild does not execute `shell` or `eval` side effects by default (Section 2.2); any file whose selection genuinely depends on such an effect is reported as an incomplete result, not silently omitted or silently assumed absent. This is a real completeness gap, not just a conservative default: a future opt-in effect boundary (a sandboxed command runner) is planned but not yet implemented, and every RQ2/RQ4 number in this paper should be read against that gap. Kconfig validity — distinguishing a condition that is merely Make-satisfiable from one a real `.config` could actually produce under Kconfig’s own dependency/select/choice rules — is not modeled at all today: skbuild reports the former, not the latter, so a reported condition can describe a configuration Kconfig itself would reject. Every evaluation number in this paper is tied to one pinned source revision per project; Kbuild conventions have historically drifted across kernel releases (see the retired `LINUX_ANALYSIS_TODO.md`’s own documented gaps between kernel versions in this project’s history), so these numbers should not be read as claims about every release of these projects.

5 Related Work

Symbolic execution of build descriptions. Nguyen and Nguyen [3] first applied symbolic execution specifically to Linux Kbuild Makefiles, treating `CONFIG_*` variables as finite-domain symbolic inputs solved with Z3; this paper is a from-scratch reimplement of that direction under an eager branch-and-merge execution strategy (Section 3) rather than that system’s own strategy. Tamrawi et al.’s SYMake [4] symbolically evaluates GNU Make itself (not specifically Kbuild) to analyze build code more generally. King’s original symbolic execution [5] underlies both lines of work, applied here to a declarative build description rather than an imperative program.

CMake. Cybolic [1] solves the structurally identical problem — per-file build condition extraction via symbolic execution — for CMake, and is the direct source of this paper’s eager branch-and-merge representation (Section 3); Section 1 discusses where Kbuild’s variability model is simpler (finite Kconfig domains) and where it is not (conditional recursive traversal). Nguyen, Nguyen, and Phan [2] posed the CMake build-condition question that Cybolic answers, as a vision piece with neither implementation nor evaluation.

Variability-aware analysis. Erwig and Walkingshaw’s choice calculus [6] provides a general representation for software variation that this paper’s guarded-value state can be understood as a build-system-specific instance of.

6 Conclusion

We presented skbuild, a symbolic-execution engine that computes, for every object file in a Kbuild-style build, a Z3-checkable Boolean condition over Kconfig symbols describing when that file is selected. Its central design decision — an eager branch-and-merge symbolic state, adapted from Cybolic’s CMake analyzer, in place of a fork-per-branch design we found does not scale to real Kbuild trees — required two Kbuild-specific correctness fixes with no CMake analogue: scoping a conditionally-named overwrite to the universe in which it actually fires, and re-gating carried-forward values by branch guard at merge time. **[TODO: headline evaluation result]**

References

- [1] Anonymous Author(s). 2027. Cybolic: Extracting CMake Build Conditions via Symbolic Execution. *Proc. ACM Softw. Eng.*, FSE (2027).
- [2] ThanhVu Nguyen, KimHao Nguyen, and Anh Cuong Phan. 2022. Analyzing the CMake Build System. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '22)*.
- [3] KimHao Nguyen and ThanhVu Nguyen. 2020. Using Symbolic Execution to Analyze Linux KBuild Makefiles. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME '20)*.

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

[4] Ahmed Tamrawi, Hoan Anh Nguyen, Hung Viet Nguyen, and Tien N. Nguyen. 2012. Build Code Analysis with Symbolic Evaluation. In *Proceedings of the 34th International Conference on Software Engineering (ICSE '12)*.

[5] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (1976).

[6] Martin Erwig and Eric Walkingshaw. 2011. The Choice Calculus: A Representation for Software Variation. *ACM Trans. Softw. Eng. Methodol.* 21, 1 (2011).